Injecting Numerical Reasoning Skills into Language Models

Mor Geva*
Tel Aviv University,
Allen Institute for AI
morgeva@mail.tau.ac.il

Ankit Gupta*
Tel Aviv University
ankitgupta.iitkanpur@gmail.com

Jonathan Berant
Tel Aviv University,
Allen Institute for AI
joberant@cs.tau.ac.il

Abstract

Large pre-trained language models (LMs) are known to encode substantial amounts of linguistic information. However, high-level reasoning skills, such as numerical reasoning, are difficult to learn from a language-modeling objective only. Consequently, existing models for numerical reasoning have used specialized architectures with limited flexibility. In this work, we show that numerical reasoning is amenable to automatic data generation, and thus one can inject this skill into pre-trained LMs, by generating large amounts of data, and training in a multi-task setup. We show that pre-training our model, GENBERT, on this data, dramatically improves performance on DROP (49.3 $\rightarrow$ 72.3 F_1), reaching performance that matches state-of-the-art models of comparable size, while using a simple and general-purpose encoder-decoder architecture. Moreover, GENBERT generalizes well to math word problem datasets, while maintaining high performance on standard RC tasks. Our approach provides a general recipe for injecting skills into large pre-trained LMs, whenever the skill is amenable to automatic data augmentation.

1 Introduction

Recently, models trained on large amounts of data with a language modeling (LM) objective, have shown great promise in natural language processing, exhibiting surprising amounts of knowledge and information (Peters et al., 2018; Devlin et al., 2019; Liu et al., 2019; Lan et al., 2019; Petroni et al., 2019; Hewitt and Manning, 2019). However, high-level skills, such as the ability to perform numerical reasoning over text, can be challenging to capture with a LM objective only. Consider the example in Table 1. To solve the first question (Q1), a model must capture the value of numbers in the text, compute their difference, and generate the tokens corresponding to the result, which generally do not appear in the input passage.

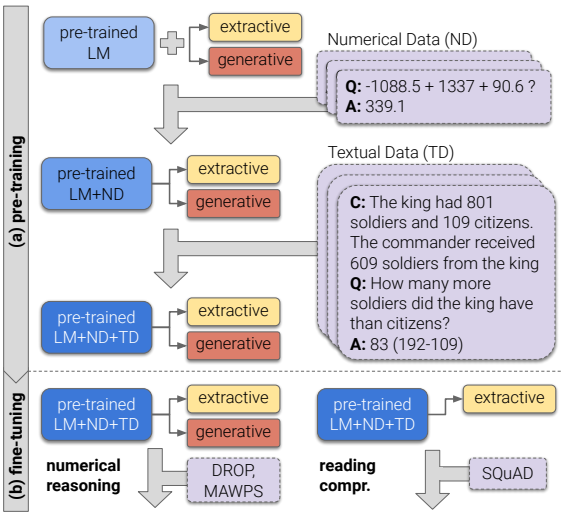

Figure 1: An overview of our approach for injecting numerical skills into a pre-trained LM. (a) We add two pre-training steps over large amounts of synthetic numerical data (ND) and textual data (TD); (b) we further fine-tune the model over either numerical reasoning datasets (DROP, MAWPS) or reading comprehension datasets (SQUAD).

To make the task more manageable, state-of-the-art models have employed specialized architectures, restricting the space of possible numerical computations to a limited set. Modules were designed for counting (but only until '9') and for addition and subtraction (but of 2-3 numbers only). Such models perform well on existing datasets, such as DROP (Dua et al., 2019), but do not generalize to unsupported computations, which will require modifying the model architecture. Moreover, current models marginalize at training time over all numerical expressions that evaluate to the correct answer. Since the number of such expressions grows exponentially, scaling these approaches to arbitrary computations entails using non-differentiable operations (sampling or computing top-K numerical expressions), which can lead to training difficulties.

*These authors contributed equally.

Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pages 946–958
July 5 - 10, 2020. ©2020 Association for Computational Linguistics

Passage: *Taunton has four art galleries...* ***Hughes/ Donahue Gallery*** *founded in* ***2007****, a local community gallery serving local Taunton artists...* ***Art Euphoric*** *founded in* ***2008*** *has both visual and craft exhibits...*
Q1: How many **years** **after** founding of **Hughes/ Donahue** was **Art Euphoric** founded? **A1**: 1 (number)
Q2: Which gallery was founded **later**, **Hughes/ Donahue** or **Art Euphoric**? **A2**: Art Euphoric (span)

Table 1: Example passage from DROP, and two questions with different answer types.

In this work, we propose that reasoning skills, such as numerical reasoning, are amenable to *automatic data generation*. Hence, one can inject that skill directly into the model by adding additional pre-training steps, allowing the model to learn the skill in an end-to-end fashion. This results in a fully-differentiable training procedure over a standard and general-purpose architecture, where the output space can be easily controlled through the data generation procedure.

Specifically (Figure 1), we add to a large pre-trained LM two pre-training steps over automatically-generated synthetic data. First, we generate *numerical data* of the form $3 + 4 + 11 = 18$. Training on this data teaches the model to compute the value of numbers from their tokens and to perform numerical operations. Second, we automatically generate question-passage pairs that require numerical reasoning using a compact grammar (*textual data*). Training on this data endows the model with the ability to understand computations expressed in pseudo-natural language.

In both pre-training steps, the model, GENBERT, generates output numbers token-by-token. Thus, the model has a standard architecture, where an answer can either be extracted from the input question and passage or generated from a decoder. Pre-training is done in a multi-task setup with a standard LM objective, in order to avoid "catastrophic forgetting" (Kirkpatrick et al., 2017) of the linguistic information in the original LM. After pre-training, the model has sufficient language and numerical skills to be directly fine-tuned on a target numerical reasoning dataset, without resorting to specialized architectures. Augmenting more numerical skills does not require changing the model, only generating additional data.

We demonstrate the validity of our approach by a series of experiments showing that:

(a) GENBERT is able to solve pre-training tasks for numerical reasoning.

(b) Pre-training on these tasks provides GENBERT with 1) skills to reach performance that matches state-of-the-art models of comparable size on DROP (Dua et al., 2019), a standard numerical reasoning dataset, as well as 2) the ability to generalize to math word problem (MWP) datasets (Koncel-Kedziorski et al., 2016).

(c) GENBERT learns these numerical skills while maintaining high performance on SQuAD (Rajpurkar et al., 2016), a standard reading comprehension dataset.

(d) Initializing models for numerical reasoning with GENBERT's weights improves their original performance.

To conclude, in this work we address the problem of injecting LMs with numerical reasoning skills. Our contributions are:

- A method for injecting skills into pre-trained LMs, given that automatic data generation is possible.
- GENBERT, an architecture for pre-trained LM with generative and extractive abilities.
- A framework for generating numerical and textual synthetic data for numerical reasoning.

Our code and data can be downloaded from `https://github.com/ag1988/injecting_numeracy`.

2 Numerical Reasoning Over Text

Numerical reasoning over text (NRoT) is commonly set up as a reading comprehension (RC) task. Given a training set of question-context-answer triples $\{(\mathbf{q}_i, \mathbf{c}_i, a_i)\}_{i=1}^{N}$, the goal is to learn a function that returns the answer a to a question $\mathbf{q}$ given a context $\mathbf{c}$. However, in NRoT the answer generally requires to internally perform some numerical computation using the entities and numbers in the context. Specifically, the answer is either: (a) a span (or list of spans) from the context $\mathbf{c}$ or question $\mathbf{q}$, or (b) a number that is the result of some computation (see examples in Table 1).

Two natural, yet opposing, approaches lend themselves to tackling NRoT: (a) *A symbolic approach*: a model can read the question and context, output a numerical expression and evaluate the answer with an external symbolic calculator. This approach is a particular case of semantic parsing (Kamath and Das, 2019), and was common in early NRoT datasets (Koncel-Kedziorski et al., 2015; Roy and Roth, 2015; Hosseini et al., 2014). How-

ever, it suffers from several drawbacks. First, because numerical expressions are discrete and their space grows combinatorially, the model must learn to search in this space using non-differentiable operations, which are usually difficult to optimize. Second, numerical expressions are limited to numerical answers, while in DROP often a numerical computation is required but the final answer is a text span. (b) *A distributed approach*: have a model directly generate the answer given $(\mathbf{q}, \mathbf{c})$. When the answer is a text span, the model can extract it from the input, and when the answer is a number that is not in $\mathbf{q}$ or $\mathbf{c}$, the model must generate it. While this makes training straightforward, the model must learn to perform numerical computations from the relatively small target dataset. We empirically show in §3 that this leads to low performance in general.

As a compromise, most NRoT models (Dua et al., 2019; Kinley and Lin, 2019; Hu et al., 2019; Efrat et al., 2019) have taken a hybrid approach: they augment standard extractive QA models with specialized modules for handling a limited set of numerical computations. We briefly describe this architecture, as it is the basis for our model in §3.

Given a question with n_1 tokens $\mathbf{q} = (q_1, \ldots, q_{n_1})$ and a context with n_2 tokens $\mathbf{c} = (c_1, \ldots, c_{n_2})$, the hybrid model first computes contextualized representations for the $n_1 + n_2 + 3$ tokens $\langle$`[CLS]` $\mathbf{q}$ `[SEP]` $\mathbf{c}$`[SEP]`$\rangle$ using a pre-trained LM, such as BERT (Devlin et al., 2019):

$$\mathbf{L} = \mathbf{LM}(\mathbf{q}, \mathbf{c}).$$

The representations $\mathbf{L}$ are then passed to multiple *heads*, which are small neural networks that estimate $p(a \mid \mathbf{q}, \mathbf{c}, h)$, that is, the probability of the answer given the input and conditioned on a head h, corresponding to a particular answer type:

- *Context span head*: computes a distribution over all spans in the *context* using a feed-forward network (FFN) $\mathbf{FF_c}(\mathbf{L})$.
- *Question span head*: computes a distribution over spans in the *question* using a FFN $\mathbf{FF_q}(\mathbf{L})$.
- *Count head*: computes a distribution over the numbers $\{0, \ldots, 9\}$ using a FFN $\mathbf{FF_{cnt}}(\mathbf{L})$.
- *Arithmetic head*: computes a distribution over all signed combinations of numbers in the context using a FFN $\mathbf{FF_{cmb}}(\mathbf{L})$ (the numbers in the context are identified in a pre-processing step).

While the first two heads are standard in extractive QA, the latter two heads are specialized and meant to handle answers that do not appear in the input.

Finally, for deciding which answer head to use for a given input, a *type* head $\mathbf{FF_{typ}}(\mathbf{L})$ outputs a probability distribution $p_{\text{head}}(h \mid \mathbf{q}, \mathbf{c})$ (using a FFN). Thus the model probability for an answer is

$$p(a \mid \mathbf{q}, \mathbf{c}) = \sum_{h \in \text{heads}} p_{\text{head}}(\text{h} \mid \mathbf{c}, \mathbf{q}) \cdot p(a \mid \mathbf{c}, \mathbf{q}, h).$$

Training is done by enumerating all of the ways in which the answer can be obtained using all of the heads, and maximizing this marginal probability.

While existing models perform well on DROP, the aforementioned architecture is not flexible. First, the output space is severely constrained – the model can only count up to '9', and numerical computations are restricted to signed combinations of a few numbers. Second, expanding the space of supported numerical computations is non-trivial, because training involves marginalizing over all expressions that lead to the correct answer. Since the space of numerical expressions grows exponentially, expanding this space quickly leads to a difficult search problem. Third, delegating numerical computations to an external symbolic calculator leads to modeling challenges, since there could be interactions between text and numerical computation: Consider the DROP question *"How many total yards did Phil Dawson throw for touchdowns?"*. Current models handle such questions by computing a sum from numbers in the text and returning the result. However, if the question was *"Who threw 45 total yards for touchdowns?"*, the model would have to compute the sum *internally*, and then find the relevant span in the text. This is impossible when the computation itself is delegated to an external calculator. Thus, training models to handle such numerical questions is desirable.

Motivated by the above arguments, we wish to push the frontier of end-to-end differentiable models for numerical reasoning. Thus, we will automatically generate large amounts of data that endow a pre-trained LM with numerical skills.

3 GENBERT: A BERT-based Model for Generating Arbitrary Outputs

We now describe a simple BERT-based generative model that performs numerical computations internally, termed GENBERT. The model combines the Transformer encoder-decoder architecture (Vaswani et al., 2017) with a pre-trained LM, specifically, BERT.

Our architecture is illustrated in Figure 2. Our encoder is a standard Transformer, initialized with